In [1]:
```
## List of imports
import numpy as np
import cirq
import cirq.google as cg
import networkx as nx
import scipy

import inspect
# visualization tools
%matplotlib inline
import matplotlib.pyplot as plt
from cirq.contrib.svg import SVGCircuit
```

Last Modified: `Monday, 08/03/2020`

Reference:

[Cirq Documentation (https://cirq.readthedocs.io/en/stable/api.html)](https://cirq.readthedocs.io/en/stable/api.html)

[Qiskit -> Cirq Interoperability (https://github.com/quantumlib/Cirq/issues/1548)](https://github.com/quantumlib/Cirq/issues/1548)

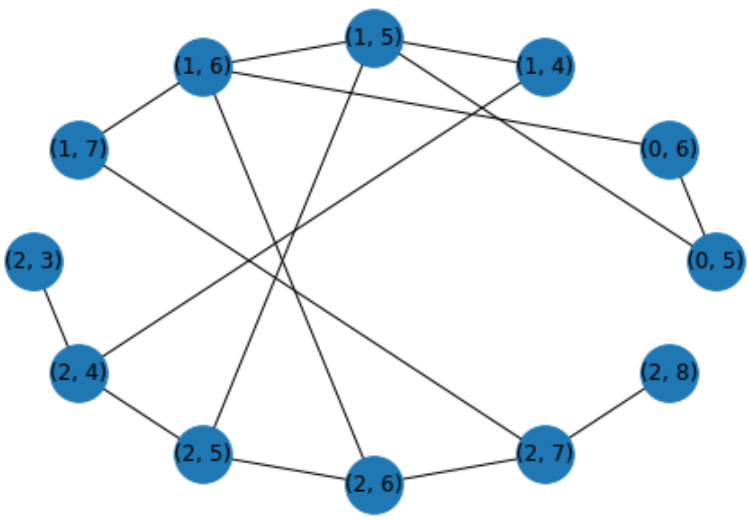
- Use `cirq.unitary` to get the unitary matrix from the circuit.
- Run the following code to optimize circuit:

  ```
  optimizer=cirq.MergeSingleQubitGates()

  optimizer.optimize_circuit(circuit)
  ```
- `random_circuit.py` mimics the implementation of its counterpart from `Qiskit`. The `cirq` implementation now works for `cirq.google.Bristlecone` and `cirq.google.Foxtail`. Usage on `cirq.Syncamore` and `cirq.Syncamore23` is pending review.
- List of available devices can be found [here (https://cirq.readthedocs.io/en/stable/google/devices.html)](https://cirq.readthedocs.io/en/stable/google/devices.html).
  - The fundamental topologies on Google devices are `cirq.LineQubit` and `cirq.GridQubit`. Connectivity is only available between adjacent qubits. The devices that can be accessed via the API are `cirq.google.Syncamore`, `cirq.google.Syncamore23`, `cirq.google.Bristlecone` and `cirq.google.Foxtail`.

Device: `Bristlecone`

Below is a dummy example that shows the connectivity of `cirq.google.Bristlecone` device and building the coupling map.

In [286]:
```
# test example for nx graphs
# get google working device
working_device = cirq.google.Bristlecone
# Identify working qubits from the device.
device_qubits = working_device.qubits
working_qubits = sorted(device_qubits)[:12]

# Populate a networkx graph with working_qubits as nodes.
working_graph = nx.Graph()
for qubit in working_qubits:
    working_graph.add_node(qubit)

# Pair up all neighbors with random weights in working_graph.
for qubit in working_qubits:
    for neighbor in working_device.neighbors_of(qubit):
        if neighbor in working_graph:
      # Generate a randomly weighted edge between them. Here the weighting
      # is a random 2 decimal floating point between 0 and 5.
            working_graph.add_edge(qubit, neighbor)

nx.draw_circular(working_graph, node_size=1000, with_labels=True);
plt.show()
```



Cirq Supported Gates

Similar to the Qiskit implementation, we list all the quantum gates that are supported by `cirq` as well as manually select 1-qubit and 2-qubit gates for the experiments in the next sections.

- In particular, as described [in the doc (https://cirq.readthedocs.io/en/latest/interop.html)](https://cirq.readthedocs.io/en/latest/interop.html), `cirq.circuits.qasm_output.QasmUGate` is sufficient for building the equivalent `Qiskit` gates `qiskit.extensions.U1Gate`, `qiskit.extensions.U2Gate` and `qiskit.extensions.U3Gate`.

In [3]:
```
# store all supported gates, can add later
# u2, u3 can be done with QasmUGate
supported_gates = set([cirq.H, cirq.I, cirq.S, \
                       cirq.T, cirq.X, cirq.Y, \
                       cirq.Z, cirq.CX, cirq.CZ, \
                       cirq.XX, cirq.YY, cirq.ZZ, \
                       cirq.rx, cirq.ry, cirq.rz, \
                       cirq.CCNOT, cirq.CCX, cirq.CCZ, \
                       cirq.CNOT, cirq.CSWAP, cirq.FREDKIN,\
                       cirq.ISWAP, cirq.SWAP, cirq.TOFFOLI,\
                       cirq.givens, cirq.riswap, \
                       cirq.CCXPowGate, cirq.CCZPowGate, \
                       cirq.CSwapGate, cirq.CZPowGate,cirq.ControlledGate,\
                       cirq.ControlledOperation, cirq.EigenGate, cirq.FSimGate,\
                       cirq.HPowGate, cirq.ISwapPowGate, cirq.PhaseGradientGate, \
                       cirq.PhasedISwapPowGate, cirq.PhasedXPowGate, cirq.PhasedXZG
ate, \
                       cirq.SwapPowGate, cirq.circuits.qasm_output.QasmUGate, cirq.
google.SYC])


## separate simple gates from parameterized gates
raw_labels = [str(gate) for gate in supported_gates]
param_labels = [str(gate) for gate in supported_gates if '<' in str(gate)]
simple_labels = [str(gate) for gate in supported_gates if str(gate) not in param_l
abels]
assert len(param_labels) + len(simple_labels) == len(raw_labels), 'double check if
all gates are included. '

# create a dictionary
label2gate_simple = {str(gate).lower(): gate for gate in supported_gates if str(ga
te) in simple_labels}
# enter manually
label2gate_param = {}
#print(str(param_labels))

# use the simple gates to test the random_circuit algorithm
test_simple_supported_gates = set([label2gate_simple[label.lower()] for label in s
imple_labels])
#print()
#print(repr(test_simple_supported_gates))
# do a simple test to get all single qubit gates
test_qubit = cirq.LineQubit(0)
test_result = []
for gate in test_simple_supported_gates:
    try:
        gate.on(test_qubit)
        test_result.append(gate)
    except Exception as e:
        test_result.append(0)
#print()
#print(test_result)
single_qubit_gates = [gate for gate in test_result if gate != 0]
# test basis gates
single_qubit_gates.append(cirq.CNOT)
single_qubit_gates.append(cirq.rx)
single_qubit_gates.append(cirq.ry)
single_qubit_gates.append(cirq.rz)

test_basis_gates = single_qubit_gates
test_basis_gates;
```

`random_circuit.py`

Below is a re-implementation of Algorithm 1 as described in the paper, via `cirq`, which works for both general case (no device used) and specifying a coupling map.

In [4]:
```
def random_circuit(n_qubits: int or None, depth: int, coupling_map=None, \
                   basis_gates=None, prob=0.5, reset=False, seed=None, quantum_dev
ice=None, \
                  see_coupling_map=False):
    """
        Trial implementation of Algorithm 1 from the reference. Generates a Random
Quantum Circuit
        according to a fed-in coupling map and a set of basis gates.

        Args:
            n_qubits:int         number of qubits on the quantum device
                * if passed in None, and if there is a QUANTUM_DEVICE, use
                all qubits from the device. Else throws error.
            depth:int               circuit depth
            coupling_map:        the topology of the qubits in the hardware
                allowed types: networkx, list

                * if there is no coupling map, we build a fully connected coupling
map
                from all the qubits
                * Graph <V,E> is specified by a set of vertices V and a set of edg
es E
                * if not nx object, coupling map can be built from an adjacency li
st
                e.g. [[1,2],[0,1]]
                * all nodes are assumed to be specified as 0-indexed integers

            basis_gates:         set of universal basis gates to construct circuit
s from
            prob:                probability of selecting CNOT
            reset:               if would like to add reset channel randomly
            seed:                random seed to be input to numpy
            quantum_device:      Google cirq device used, default is None.

        Returns:
            rqc:              random quantum circuit generated from this coupling
 map

        Raises:
            ValueError: when certain numerical inputs do not conform to the assump
tions
        from the reference, or quantum assumptions in general.

        Dependence:
            cirq, networkx, numpy
    """
    ## preliminary setup
    if reset:
        if basis_gates is None:
            raise ValueError("please input a basis set of gates. ")
        else:
            # adds the reset channel
            basis_gates.append(cirq.reset)
    if seed is None:
        np.random.seed(np.random.randint(0, 100))
    else:
        np.random.seed(seed)
    # Working device setup
    if quantum_device is not None:
        working_device = quantum_device
        if n_qubits is None:
            n_qubits = len(working_device.qubits)
        if n_qubits > len(working_device.qubits):
```

```
            raise ValueError('n_qubits exceeds the number of qubits allowed on the
device.')
        working_qubits = sorted(working_device.qubits)[:n_qubits]
        # graph with working_qubits as nodes.
        coupling_map = nx.Graph()
        for qubit in working_qubits:
            coupling_map.add_node(qubit)
        # Pair all neighbors in working_graph.
        for qubit in working_qubits:
            for neighbor in working_device.neighbors_of(qubit):
                if neighbor in coupling_map:
                    coupling_map.add_edge(qubit, neighbor)
    else:
        # no device specified, in test mode, create grid
        if n_qubits is None:
            raise ValueError('No working device found, ' +\
                             'please input a working device or specify number of t
est qubits.')
        grid = [cirq.GridQubit(i+1,0) for i in range(n_qubits)]
        working_qubits = sorted(grid)
        if coupling_map is None:
            coupling_map = nx.complete_graph(grid)
        elif isinstance(coupling_map, list):
            assert len(coupling_map) != 0, 'must specify adjacency'
            graph = nx.Graph()
            for source, target in coupling_map:
                graph.add_edge(source, target)
            coupling_map = graph
    # use all available gates if basis gates is not specified
    if basis_gates is None:
        basis_gates = test_basis_gates
    if see_coupling_map:
        nx.draw(coupling_map, with_labels=True)
    ## begin assemblying random circuit
    rqc = cirq.Circuit()
    for _ in range(depth):
        working_graph = coupling_map.copy()
        while working_graph.number_of_nodes() != 0:
            if np.random.rand() <= prob and working_graph.number_of_edges() != 0:
                operator = cirq.CNOT
                idx = np.random.choice(range(working_graph.number_of_edges()))
                operands = list(working_graph.edges)[idx]
            else:
                basis_gates_tmp = basis_gates.copy()
                # exclude CNOT
                basis_gates_tmp.remove(cirq.CNOT)
                idx = np.random.choice(range(len(basis_gates_tmp)))
                operator = list(basis_gates_tmp)[idx]
                # choose qubit
                idx = np.random.choice(range(working_graph.number_of_nodes()))
                operands = list(working_graph.nodes)[idx]
            ## apply gate operations and append to circuit
            try:
                if type(operands) == tuple:
                    operands_list = list(operands)
                else:
                    operands_list = [operands]
                if len(operands_list) == 1:
                    q1 = operands_list[0]
                    if operator not in test_simple_supported_gates:
                        ## here uses an adaptive algorithm to sample
                        #  angles depending on the number of params the
                        #  gate takes.
                        done = False
```

```
                        param_length = 1
                        while not done:
                            random_params = np.random.uniform(0, 2*np.pi, param_le
ngth)
                            try:
                                operator = operator(*random_params)
                                done = True
                                break
                            except Exception:
                                #print("adding param length, resampling ...")
                                param_length = param_length + 1
                                continue
                        assert 'operator' in locals(), 'No operator found. '
                    rqc.append([operator.on(q1)], strategy=cirq.circuits.InsertStr
ategy.INLINE)
                    working_graph.remove_node(q1)

                elif len(operands_list) == 2:
                    # two qubit operator
                    q1, q2 = operands_list
                    rqc.append([operator.on(q1, q2)], strategy=cirq.circuits.Inser
tStrategy.INLINE)
                    working_graph.remove_edge(q1, q2)
                    working_graph.remove_node(q1)
                    working_graph.remove_node(q2)
            except Exception:
                raise ValueError("Gate application failed. Printing the error mess
age above.")
    return rqc
```

Below shows a simple test that the spectrum of the block-encoded matrix is reasonably spread out using a modest RACBEM circuit depth. The section following contains 2 dummy unit tests ensuring that the random circuit algorithm is generating the expected output (with moderate depths for both cases with and without device, and displaying the qubits randomly chosen).

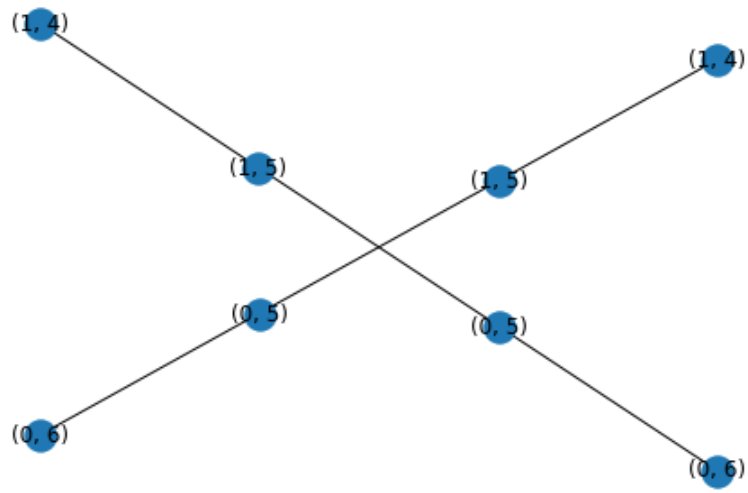
In [85]:
```
# testing singular value
total_qubit = 5
sys_qubit   = 3
total_depth = 15

test_qc = random_circuit(total_qubit, total_depth, None, None, 0.5, False, None)
UA = test_qc.unitary()
A  = UA[0:2**sys_qubit, 0:2**sys_qubit]
(svd_U, svd_S, svd_VH) = scipy.linalg.svd(A)
print("singular values of Block-Encoded matrix using 3 system qubits and 2 block-e
ncoding qubits.\n", svd_S)
```

```
singular values of Block-Encoded matrix using 3 system qubits and 2 block-encodin
g qubits.
 [0.74247728 0.74247728 0.52397947 0.52397947 0.35278999 0.35278999
 0.22267218 0.22267218]
```

In [90]:
```
import unittest
class DummyTestNoDevice(unittest.TestCase):
    def setUp(self):
        self.my_random_circuit_no_device \
        = random_circuit(4, 6, None, None, 0.5, False, 10, see_coupling_map=False)
    def test_attribute_num_qubit(self):
        self.assertEqual(len(self.my_random_circuit_no_device.all_qubits()), \
                         4, "number of qubits used should match. ")
    def test_circuit_depth(self):
        self.assertEqual(len(self.my_random_circuit_no_device.moments), 6, \
                        "need to restrain circuit depth. ")

class DummyTestWithDevice(unittest.TestCase):
    def setUp(self):
        self.google_device = cirq.google.Sycamore
        self.num_qubit = 4
        self.my_rqc_w_device \
        = random_circuit(self.num_qubit, 6, None, None, \
                         0.5, None, None, cirq.google.Bristlecone, see_coupling_ma
p=True)
        self.my_rqc_w_device_free \
        = random_circuit(None, 6, None, None, \
                         0.5, None, None, cirq.google.Bristlecone, see_coupling_ma
p=False)

    def test_attribute_num_qubit(self):
        self.assertEqual(len(self.my_rqc_w_device.all_qubits()), self.num_qubit, \
                         "specified number of qubits to use. ")

    def test_attribute_num_qubit_free(self):
        self.assertNotEqual(len(self.my_rqc_w_device_free.all_qubits()), self.num_
qubit, \
                         "did not specify number of qubits to use. ")

suite1 = unittest.TestLoader().loadTestsFromTestCase(DummyTestNoDevice)
suite2 = unittest.TestLoader().loadTestsFromTestCase(DummyTestWithDevice)
suite_final = unittest.TestSuite([suite1, suite2])
# for manual testing: unittest.main(argv=['first-arg-is-ignored'], exit=False)
unittest.TextTestRunner(verbosity=2).run(suite_final)
```

```
test_attribute_num_qubit (__main__.DummyTestNoDevice) ... ok
test_circuit_depth (__main__.DummyTestNoDevice) ... ok
test_attribute_num_qubit (__main__.DummyTestWithDevice) ... /opt/anaconda3/lib/py
thon3.7/site-packages/networkx/drawing/nx_pylab.py:579: MatplotlibDeprecationWarn
ing:
The iterable function was deprecated in Matplotlib 3.1 and will be removed in 3.
3. Use np.iterable instead.
  if not cb.iterable(width):
ok
test_attribute_num_qubit_free (__main__.DummyTestWithDevice) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.392s

OK
```

Out[90]: <unittest.runner.TextTestResult run=4 errors=0 failures=0>



QSP Circuit

The following section contains the main script for generating a QSP circuit as described in the QSVT paper (https://arxiv.org/pdf/1806.01838.pdf). The QSPCircuit class currently only supports 2 ancilla qubits, and the following instance methods:

- `build_alternating_module` : constructs a sub-circuit implementing the alternating phased sequence U_Φ, as described in Definition 15 and Lemma 19 of the paper. The circuit is constructed by alternating $U, U^\dagger$ gate applications and the controlled-Π projection gates.
- `build_controlled_projection` : constructs a controlled-Π projection gate $e^{i\phi(2\Pi - I)} = C_\Pi NOT(I \otimes Rz(\phi))C_\Pi NOT$, as decribed in Lemma 18 of the paper.

  Setting $\Pi = |0^m\rangle\langle 0^m| \otimes I$, we can use 1 extra signal qubit to implement the QSP circuit.

In [189]:
```
## classes
class QSPCircuit():
    """Build a QSP circuit.

    Attributes:
        qspc: QSP circuit.
        qregs: quantum registers

    Review Notice:
        * add in device case later, all qubits are arbitrary GridQubit now
        * consider more than 2 ancilla qubits for block-encoding

    """
    def __init__(self, n_signal_qubit, n_be_qubit, n_system_qubit):
        assert n_be_qubit <= 2, 'Only supports at most 2 ancilla qubits. '
        self.n_signal_qubit = n_signal_qubit
        self.n_be_qubit = n_be_qubit
        self.n_system_qubit = n_system_qubit
        self.n_total_qubits = sum([self.n_signal_qubit, self.n_be_qubit, self.n_sy
stem_qubit])
        ## TODO: add in device case later
        self.qubit_grid = [cirq.GridQubit(x, 0) for x in range(self.n_total_qubits
)]
        self.signal_qubits = self.qubit_grid[0:self.n_signal_qubit]
        self.be_qubits = self.qubit_grid[self.n_signal_qubit:self.n_signal_qubit+s
elf.n_be_qubit]
        self.qspc = cirq.Circuit()


    # circuit generating methods below for QSP, refer to QSVT paper Definition 15-
Lemma 19
    # Definition 15: phase alternating sequence
    # Corollary 18: SVT by real polynomials
    def build_alternating_module(self, be_qc, phi_seq, real_part=True, measure=Fal
se, init_prepare=None):
        """ Creates the main circuit for Quantum Signal Processing as defined in
            Lemma 18 of [ref: QSVT].

            Args:
                be_qc,              a BlockEncoding quantum circuit that
                                    generates the encoding operator U

                phi_seq,            the set of control parameters \phi used in QSP

                real_part,          if True, encode the real-part of the polynomial
using QSP,
                                    consistent with Corollary 18.

                measure,            if True, adds measurement gates to the ancilla
 qubits (signal qubits
                                    and BE qubits)

        """
        qubit_grid = self.qubit_grid.copy()
        signal_qubit = qubit_grid[0]
        qsp_circuit = cirq.Circuit()

        if real_part:
            # Hadamard gate creates uniform superposition between the 2 possible
            # states, (P + P*)/2 is the real part
            qsp_circuit.append(cirq.H.on(signal_qubit))

        # flag to append U_dag instead of U
```

```
        dag = False
        qc_crot = self.build_controlled_projection(phi_seq[-1])
        qsp_circuit.append(qc_crot)
        for phi in reversed(phi_seq[:-1]):
            if not dag:
                qsp_circuit.append(be_qc)
            else:
                qsp_circuit.append(cirq.inverse(be_qc))

            if real_part:
                # Add a Z gate before the control.
                qsp_circuit.append(cirq.Z(qubit_grid[0]))

            qc_crot = self.build_controlled_projection(phi)
            qsp_circuit.append(qc_crot)
            dag = not dag
        if real_part:
            # add another Hadamard gate at the end for real part
            qsp_circuit.append(cirq.H.on(signal_qubit), cirq.InsertStrategy.NEW)

        if measure:
            # add measurement gates to the be qubits and signal qubits
            for q in self.be_qubits + self.signal_qubits:
                qsp_circuit.append(cirq.measure(q))
        self.qspc = qsp_circuit

    def build_controlled_projection(self, phi):
        """ Returns a controlled-Pi gate application on a block-encoding.
            As described in the reference: QSVT, RACBEM, adds 1 or 2 signal qubit
(s)
            and applies a controlled-Projection (controlled on 0).

            Args:
                phi,                        phase factor, parameter passed into Rz g
ate

        """
        qr = [cirq.GridQubit(x, 0) for x in range(self.n_signal_qubit + self.n_be_
qubit)]
        qc = cirq.Circuit()

        # Add an X gate to flip control to 0.
        if self.n_be_qubit == 1:
            instructions = [cirq.X(qr[1]), cirq.CX(qr[1], qr[0]), \
                           cirq.rz(phi*2.).on(qr[0]), cirq.CX(qr[1], qr[0]), \
                            cirq.X(qr[1])]
            qc.append(instructions)
        elif self.n_be_qubit == 2:
            instructions = [cirq.X(qr[1]), cirq.X(qr[2]), cirq.CCX(qr[1], qr[2], q
r[0]), \
                            cirq.rz(phi * 2.).on(qr[0]), cirq.CCX(qr[1], qr[2], qr
[0]), \
                            cirq.X(qr[1]), cirq.X(qr[2]) ]
            qc.append(instructions)
        else:
            raise ValueError("currently only supports 2 ancilla qubits. ")

        return qc
```

Block Encoding Circuit

The section below contains the implementation of RACBEM and H-RACBEM from the reference (https://arxiv.org/pdf/2006.04010.pdf). The generic RACBEM uses `random_circuit` to randomly select gate applications of a desired circuit depth from a `basis_gates` set; whereas the H-RACBEM code uses the QSP circuit from the previous section to ensure the encoded matrix is Hermitian.

- To view the encoded matrix, simply call `<cirq.Circuit()>.unitary()`, and slice the block representing our system qubits. The matrix simulated by the entire circuit is unitary, but the block encoded part is not.

In [190]:
```
## Implementation of RACBEM
"""Module for RACBEM implemented using Google's Cirq library.

This is an implementation of the RAndom Circuit Block Encoded Matrix
(RACBEM) and its Hermitian conjugate. It is then used to build a quantum
singular value circuit using the method of quantum singular value
transformation (QSVT).

Take a RAndom Circuit Block Encoded Matrix (RACBEM), this function uses
a quantum signal processing circuit to evaluate the matrix inverse,
using the method of quantum singular value transformation (QSVT). This
implements a (non-Hermitian) block-encoding of a Hermitian matrix
manually.

Notes:
    Will consider scaled noise model later.
    ...

References:
    Yulong Dong, Lin Lin. Random circuit block-encoded matrix and a proposal
        of quantum LINPACK benchmark.
Authors:

Version: 1.0
Last revision: 08/04/2020"""

class BlockEncoding():
    """Block encoding implementation on Cirq.

       Any N-qubit non-unitary matrix can be block-encoded by using a sufficiently
       general (N+1)-qubit quantum circuit and measuring the ancilla bit.
    """
    def __init__(self, n_be_qubit, n_system_qubit):
        self.n_sys_qubit = n_system_qubit
        self.n_be_qubit = n_be_qubit
        self.n_total_qubit = self.n_sys_qubit + self.n_be_qubit
        ## TODO: generalize to device case
        self.qubit_grid = [cirq.GridQubit(x, 0) for x in range(self.n_total_qubit
)]

    def build_random_circuit(self, depth, coupling_map=None, \
                             basis_gates=None, prob=0.5, reset=False, seed=None, \
                             quantum_device=None, see_coupling_map=False):
        """ Generates a random circuit using Algorithm 1. """
        if not hasattr(self, 'random_circuit'):
            self.random_circuit = random_circuit(self.n_total_qubit, depth, coupli
ng_map, \
                                                 basis_gates, prob, reset, seed, qu
antum_device, see_coupling_map)

    def build_dag(self):
        """ Builds circuit for U_A dagger by inverting the reversible gates in the
circuit.

            Returns True/False signaling whether build was successful.

        """
        if hasattr(self, 'random_circuit'):
            self.random_circuit_inv = cirq.inverse(self.random_circuit)
            return True
        else:
            return False
```

```
    def build_measurement(self):
        """ Returns a copy of the original block encoding with measurement gate ad
ded on the
            1st qubit for Block-encoding.

            Review notice:
                * currently only measures the 1st qubit in sorted order.
        """
        assert hasattr(self, random_circuit), "must have generated a random circui
t before measurement. "
        assert hasattr(self, qreg), "must have generated a qubit grid to operate o
n; " + \
                "try running build_random_circuit "
        rc_copy = self.random_circuit.copy()
        rc_copy.append(cirq.measure(self.qreg[0]))
        return rc_copy

class Hermitian_BlockEncoding(BlockEncoding):
    """Implementation of a Hermitian-RACBEM on Cirq.
    """
    def __init__(self, n_be_qubit, n_system_qubit):
        assert n_be_qubit >= 2, 'must have at least 2 ancilla qubits in order to b
uild canonical H-racbem. '
        BlockEncoding.__init__(self, n_be_qubit, n_system_qubit)
        self.qsp_circuit = QSPCircuit(1, n_be_qubit-1, n_system_qubit)

    def build_random_circuit(self, depth, coupling_map=None, \
                             basis_gates=None, prob=0.4, reset=True, seed=None, \
                             quantum_device=None, see_coupling_map=False, canonical
=False):
        """
            Hermitian BlockEncoding circuit

            Args:
                BlockEncoding::build_random_circuit(*kwargs)

                canonical,            if True, generates a canonical H-RACBEM (usi
ng the result from
                                      reference:figure17)
        """
        assert hasattr(self, 'phi_seq'), "needs phi parameters for QSP circuit. "
        if canonical:
            assert self.n_be_qubit == 2, "only supporting 2 BlockEncoding qubits.
 "
        rqc = random_circuit(self.n_total_qubit-1, depth, \
                                          coupling_map=coupling_map, basis_gates
=basis_gates, prob=prob)
        if canonical:
            # canonical H-RACBEM
            # the 2 BE qubits
            q1, q2 = self.qubit_grid[0], self.qubit_grid[1]
            canonical_circuit = cirq.Circuit()
            canonical_circuit.append([cirq.H(q1), cirq.T(q1)])
            canonical_circuit.append(rqc)
            canonical_circuit.append(cirq.CNOT(q2, q1))
            # note, S-dagger
            canonical_circuit.append((cirq.S**(-1)).on(q1))
            canonical_circuit.append(cirq.CNOT(q2, q1))
            canonical_circuit.append(cirq.inverse(rqc))
            canonical_circuit.append(cirq.T(q1))
            canonical_circuit.append(cirq.H(q1))
            self.random_circuit = canonical_circuit.copy()
        else:
            self.qsp_circuit.build_alternating_module(rqc, self.phi_seq, \
```

```
                                                                real_part=True, measure=Fals
e, init_prepare=None)
            self.random_circuit = self.qsp_circuit.qspc.copy()

    def set_condition_number(self, kappa):
        """ Condition number of H-RACBEM is controlled by the second order poly
            h_{\kappa}(x) = (1-kappa**(-1))x**2 + kappa**(-1)
        """
        a, b = 1 - kappa**(-1), kappa**(-1)
        self.set_ab(a, b)

    def set_ab(self, a, b):
        """Construct phi_seq according to h(x) = a*x^2+b"""
        self.ab = (a, b)
        phi0 = 0.25 * (np.arccos(a+b) + np.arccos(b))
        phi1 = 0.5 * (np.arccos(a+b) - np.arccos(b))
        self.phi_seq = np.array([phi0,phi1,phi0])
```

Integration Test

Below is a simple integration test mimicking the IBM Qiskit version. Using a small `basis_gates` set that only contains `CNOT, Rx, Ry, Rz`, generate RACBEM and H-RACBEM with depth 15.

- Using stored phase factors, verify that the QSVT circuit can generate an approximation to the matrix inverse A^{-1}, by multiplying the generated matrix $g(\aleph)$ with a test vector (here simply used $|b\rangle = |0^n\rangle$), sample the output (assuming Monte Carlo noise is the only source), and compute the success probability, which should be given by the bound below:

$$p_{succ} = \|g(\aleph)\,|b\rangle\,\|_2^2 \geq (\frac{1}{\kappa} - \epsilon)^2$$

  - Generate the final state from the circuit "analytically" without noise, and also run the circuit assuming there is Monte Carlo noise from measurement.
- For the H-RACBEM, verify that the condition number of the encoded random Hermitian matrix:
  - has tunable condition number (here used $\kappa = 5$).
  - is indeed Hermitian, by checking $\|U - U^\dagger\|_2 = 0$.

In [252]:
```
# integration test: main.py, noiseless
kappa = 5                    # condition number
n_sys_qubit = 3              # the number of system qubits
n_be_qubit = 1               # the number of block-encoding qubit
n_sig_qubit = 1              # the number of signal qubit
n_tot_qubit = n_sig_qubit+n_be_qubit+n_sys_qubit
n_depth = 15                 # the depth of random circuit
prob_one_q_op = 0.5          # the probability of selecting a one-qubit
                             # operation when two_q_op is allowed
basis_gates = [cirq.CX, cirq.rx, cirq.ry, cirq.rz]

simulation_reps = 130       # number of simulations to compute succ prob with noise
sigma = 1.0                  # parameter used to rescale noise model
# test state \ket{0^n}
b = np.zeros([2**n_sys_qubit,])
b[0] = 1.0

# instances of RACBEM
be = BlockEncoding(n_be_qubit, n_sys_qubit)
qsp = QSPCircuit(n_sig_qubit, n_be_qubit, n_sys_qubit)

### TODO: add device architecture, rescale noise model
# Google engine?

###
be.build_random_circuit(n_depth, None, basis_gates=basis_gates, prob=prob_one_q_op
)
be.build_dag()

# get block-encoded matrix
UA = be.random_circuit.unitary()
A = UA[0:2**n_sys_qubit, 0:2**n_sys_qubit]
# do SVD and verify matrix
svd_U, svd_S, svd_VH = scipy.linalg.svd(A)
print("singular value BlockEncoded (A) = \n", np.around(svd_S, decimals=3))

# load phase factors, normally need to run Remez.ipynb, here
# only using an example
data = np.loadtxt("phi_inv_%d.txt"%(kappa))
phi_seq = data[:-2]
scale_fac = data[-2]

# run simulation and do measurements
qsp.build_alternating_module(be.random_circuit, phi_seq, real_part=True, measure=T
rue)
cirq_simulator = cirq.Simulator()
simulation_data = cirq_simulator.run(qsp.qspc, repetitions=simulation_reps)

# process and get all measurement data, count '|00>'
simulation_dataframe = simulation_data.data
strdata_q1 = simulation_dataframe[simulation_dataframe.columns[0]].values.astype(
'str')
strdata_q2 = simulation_dataframe[simulation_dataframe.columns[1]].values.astype(
'str')
measurement_str_data = np.char.add(strdata_q1, strdata_q2)
# count occurrences
unique, counts = np.unique(measurement_str_data, return_counts=True)
result_dict = dict(zip(unique, counts))
print("succ prob (measured)  = ", result_dict['00']/simulation_reps)

# qsp testing
qsp.build_alternating_module(be.random_circuit, phi_seq, real_part=True, measure=F
alse)
```

```
# simulate final state using cirq
cirq_simulator = cirq.Simulator()
state = cirq_simulator.simulate(qsp.qspc).final_state

x = state[0:2**n_sys_qubit]
prob_qsp = scipy.linalg.norm(x)**2
# exact success prob
svd_S_herm = (1-1.0/kappa)*svd_S**2+1.0/kappa
A_herm_inv = svd_VH.transpose().conjugate() @ np.diag(1/svd_S_herm) @ svd_VH
x_exact = A_herm_inv @ b / scale_fac
prob_exact = scipy.linalg.norm(x_exact)**2
print("succ prob (exact)     = ", prob_exact)
print("succ prob (noiseless) = ", prob_qsp)

## Hermitian BE testing
n_be_qubit += 1
be = Hermitian_BlockEncoding(n_be_qubit, n_sys_qubit)

be.set_condition_number(kappa)
phi_0, phi_1 = be.phi_seq[0], be.phi_seq[1]

be.build_random_circuit(n_depth, None, basis_gates=basis_gates, \
                        prob=0.6)
be.build_dag()
UA = be.random_circuit.unitary()
A = UA[0:2**n_sys_qubit, 0:2**n_sys_qubit]
UA = cirq.inverse(be.random_circuit).unitary()
A_dag = UA[0:2**n_sys_qubit, 0:2**n_sys_qubit]
svd_U, svd_S, svd_VH = scipy.linalg.svd(A)
print("Hermitian RACBEM")
print("singular value (A) = \n", np.around(svd_S, decimals=3))
print("theo  bound condition number (A)  = {}".format(np.cos(2*phi_0+phi_1)/np.cos
(2*phi_0-phi_1)))
print("actual      condition number (A)  = {}".format(svd_S.max()/svd_S.min()))
print("||A - A^\dagger||_2             = {}".format(scipy.linalg.norm(A - A_dag)))

# processing measurement data
simulation_dataframe = simulation_data.data
strdata_q1 = simulation_dataframe[simulation_dataframe.columns[0]].values.astype(
'str')
strdata_q2 = simulation_dataframe[simulation_dataframe.columns[1]].values.astype(
'str')
measurement_str_data = np.char.add(strdata_q1, strdata_q2)
unique, counts = np.unique(measurement_str_data, return_counts=True)
dict(zip(unique, counts))
```

```
singular value BlockEncoded (A) =
 [0.981 0.956 0.899 0.851 0.79  0.631 0.476 0.371]
succ prob (measured)  =  0.07692307692307693
succ prob (exact)     =  0.11469870266747213
succ prob (noiseless) =  0.11768485859933264
Hermitian RACBEM
singular value (A) =
 [0.261 0.258 0.248 0.242 0.221 0.214 0.206 0.202]
theo  bound condition number (A)  = 4.9999999999999964
actual      condition number (A)  = 1.2914318456828349
||A - A^\dagger||_2             = 1.2789927509562018e-15
```

Out[252]:
```
{'00': 10, '01': 80, '11': 40}
```

Scale Noise Model

In [257]:
```
import numpy as np
import scipy.linalg as la

from qiskit import execute
from qiskit import Aer
from qiskit import IBMQ
from qiskit.tools.monitor import job_monitor
from qiskit.providers.aer.noise import NoiseModel

import os
import pickle

backend_name = 'ibmq_burlington'
backend = Aer.get_backend('qasm_simulator')
noise_backend = pickle.load(open(backend_name+'_backend_config.pkl','rb'))
noise_model = NoiseModel.from_dict(noise_backend['noise_dict'])
coupling_map = noise_backend['coupling_map']
tot_q_device = noise_backend['tot_q_device']
print("load architecture locally at: %s_backend_config.pkl\n"%(backend_name))
noise_backend
```

```
load architecture locally at: ibmq_burlington_backend_config.pkl
```

Out[257]:
```
{'noise_dict': {'errors': [{'type': 'qerror',
                 'operations': ['id'],
                 'instructions': [[{'name': 'id', 'qubits': [0]}],
                  [{'name': 'z', 'qubits': [0]}],
                  [{'name': 'reset', 'qubits': [0]}]],
                 'probabilities': [0.999096650168169,
                  0.0002567101928673817,
                  0.0006466396389637197],
                 'gate_qubits': [[0]]},
                {'type': 'qerror',
                 'operations': ['id'],
                 'instructions': [[{'name': 'kraus',
                    'qubits': [0],
                    'params': [array([[ 0.00872943+0.j,  0.        +0.j],
                            [ 0.        +0.j, -0.00873318+0.j]]),
                     array([[0.        +0.j, 0.02931958+0.j],
                            [0.        +0.j, 0.        +0.j]]),
                     array([[-0.9999619 -0.j,  0.        +0.j],
                            [ 0.        +0.j, -0.99953194+0.j]])]}]],
                 'probabilities': [1.0],
                 'gate_qubits': [[1]]},
                {'type': 'qerror',
                 'operations': ['id'],
                 'instructions': [[{'name': 'kraus',
                    'qubits': [0],
                    'params': [array([[ 0.01025111+0.j,  0.        +0.j],
                            [ 0.        +0.j, -0.01025473+0.j]]),
                     array([[0.        +0.j, 0.02656625+0.j],
                            [0.        +0.j, 0.        +0.j]]),
                     array([[-0.99994746-0.j,  0.        +0.j],
                            [ 0.        +0.j, -0.99959446+0.j]])]}]],
                 'probabilities': [1.0],
                 'gate_qubits': [[2]]},
                {'type': 'qerror',
                 'operations': ['id'],
                 'instructions': [[{'name': 'kraus',
                    'qubits': [0],
                    'params': [array([[ 0.00875953+0.j,  0.        +0.j],
                            [ 0.        +0.j, -0.00876213+0.j]]),
                     array([[0.       +0.j, 0.0243456+0.j],
                            [0.       +0.j, 0.       +0.j]]),
                     array([[0.99996163+0.j, 0.        +0.j],
                            [0.        +0.j, 0.9996652 +0.j]])]}]],
                 'probabilities': [1.0],
                 'gate_qubits': [[3]]},
                {'type': 'qerror',
                 'operations': ['id'],
                 'instructions': [[{'name': 'id', 'qubits': [0]}],
                  [{'name': 'z', 'qubits': [0]}],
                  [{'name': 'reset', 'qubits': [0]}]],
                 'probabilities': [0.9989600160481474,
                  0.00029998612086222694,
                  0.0007399978309903243],
                 'gate_qubits': [[4]]},
                {'type': 'qerror',
                 'operations': ['u2'],
                 'instructions': [[{'name': 'id', 'qubits': [0]}],
                  [{'name': 'z', 'qubits': [0]}],
                  [{'name': 'reset', 'qubits': [0]}]],
                 'probabilities': [0.999096650168169,
                  0.0002567101928673817,
                  0.0006466396389637197],
                 'gate_qubits': [[0]]},
```

```
{'type': 'qerror',
 'operations': ['u2'],
 'instructions': [[{'name': 'kraus',
    'qubits': [0],
    'params': [array([[ 0.00872943+0.j,  0.        +0.j],
            [ 0.        +0.j, -0.00873318+0.j]]),
     array([[0.        +0.j, 0.02931958+0.j],
            [0.        +0.j, 0.        +0.j]]),
     array([[-0.9999619 -0.j,  0.        +0.j],
            [ 0.        +0.j, -0.99953194+0.j]])]}]],
 'probabilities': [1.0],
 'gate_qubits': [[1]]},
{'type': 'qerror',
 'operations': ['u2'],
 'instructions': [[{'name': 'kraus',
    'qubits': [0],
    'params': [array([[ 0.01025111+0.j,  0.        +0.j],
            [ 0.        +0.j, -0.01025473+0.j]]),
     array([[0.        +0.j, 0.02656625+0.j],
            [0.        +0.j, 0.        +0.j]]),
     array([[-0.99994746-0.j,  0.        +0.j],
            [ 0.        +0.j, -0.99959446+0.j]])]}]],
 'probabilities': [1.0],
 'gate_qubits': [[2]]},
{'type': 'qerror',
 'operations': ['u2'],
 'instructions': [[{'name': 'kraus',
    'qubits': [0],
    'params': [array([[ 0.00875953+0.j,  0.        +0.j],
            [ 0.        +0.j, -0.00876213+0.j]]),
     array([[0.       +0.j, 0.0243456+0.j],
            [0.       +0.j, 0.       +0.j]]),
     array([[0.99996163+0.j, 0.        +0.j],
            [0.        +0.j, 0.9996652 +0.j]])]}]],
 'probabilities': [1.0],
 'gate_qubits': [[3]]},
{'type': 'qerror',
 'operations': ['u2'],
 'instructions': [[{'name': 'id', 'qubits': [0]}],
  [{'name': 'z', 'qubits': [0]}],
  [{'name': 'reset', 'qubits': [0]}]],
 'probabilities': [0.9989600160481474,
  0.00029998612086222694,
  0.0007399978309903243],
 'gate_qubits': [[4]]},
{'type': 'qerror',
 'operations': ['u3'],
 'instructions': [[{'name': 'id', 'qubits': [0]}],
  [{'name': 'z', 'qubits': [0]}],
  [{'name': 'reset', 'qubits': [0]}]],
 'probabilities': [0.9981941822773795,
  0.0005129565875156641,
  0.0012928611351047392],
 'gate_qubits': [[0]]},
{'type': 'qerror',
 'operations': ['u3'],
 'instructions': [[{'name': 'kraus',
    'qubits': [0],
    'params': [array([[ 0.01233949+0.j,  0.        +0.j],
            [ 0.        +0.j, -0.01235011+0.j]]),
     array([[0.        +0.j, 0.04145524+0.j],
            [0.        +0.j, 0.        +0.j]]),
     array([[-0.99992387-0.j,  0.        +0.j],
            [ 0.        +0.j, -0.99906403+0.j]])]}]],
```

```
     'probabilities': [1.0],
     'gate_qubits': [[1]]},
    {'type': 'qerror',
     'operations': ['u3'],
     'instructions': [[{'name': 'kraus',
       'qubits': [0],
       'params': [array([[ 0.01449138+0.j,  0.        +0.j],
               [ 0.        +0.j, -0.01450161+0.j]]),
        array([[0.        +0.j, 0.03756372+0.j],
               [0.        +0.j, 0.        +0.j]]),
        array([[0.99989499+0.j, 0.        +0.j],
               [0.        +0.j, 0.99918901+0.j]])]}]],
     'probabilities': [1.0],
     'gate_qubits': [[2]]},
    {'type': 'qerror',
     'operations': ['u3'],
     'instructions': [[{'name': 'kraus',
       'qubits': [0],
       'params': [array([[ 0.0123837 +0.j,  0.        +0.j],
               [ 0.        +0.j, -0.01239105+0.j]]),
        array([[0.        +0.j, 0.03442477+0.j],
               [0.        +0.j, 0.        +0.j]]),
        array([[0.99992332+0.j, 0.        +0.j],
               [0.        +0.j, 0.99933047+0.j]])]}]],
     'probabilities': [1.0],
     'gate_qubits': [[3]]},
    {'type': 'qerror',
     'operations': ['u3'],
     'instructions': [[{'name': 'id', 'qubits': [0]}],
      [{'name': 'z', 'qubits': [0]}],
      [{'name': 'reset', 'qubits': [0]}]],
     'probabilities': [0.9979212036545877,
      0.0005993482802214365,
      0.001479448065190847],
     'gate_qubits': [[4]]},
    {'type': 'qerror',
     'operations': ['cx'],
     'instructions': [[{'name': 'kraus',
       'qubits': [0, 1],
       'params': [array([[ 0.00000000e+00+0.j,  1.18067112e-03+0.j, -5.88286818e-
20+0.j,
                 0.00000000e+00+0.j],
               [-2.38957425e-19+0.j, -1.06741989e-32+0.j,  0.00000000e+00+0.j,
                -5.85847203e-20+0.j],
               [ 1.10133253e-16+0.j, -9.83623757e-16+0.j,  4.23691735e-33+0.j,
                -1.18338413e-03+0.j],
               [-3.92809103e-16+0.j, -1.19347709e-40+0.j,  0.00000000e+00+0.j,
                 6.35640428e-33+0.j]]),
        array([[-9.47968196e-04-0.j, -6.63921211e-77+0.j,  9.05149915e-49+0.j,
                 0.00000000e+00+0.j],
               [-2.23570963e-16+0.j,  9.49612117e-04+0.j,  0.00000000e+00+0.j,
                -9.06764332e-49+0.j],
               [-4.21832005e-19+0.j,  2.12295620e-64+0.j,  9.50146490e-04+0.j,
                 4.21486652e-77+0.j],
               [-5.63300044e-82+0.j,  1.22459058e-64+0.j,  0.00000000e+00+0.j,
                -9.51794189e-04+0.j]]),
        array([[ 0.00000000e+00+0.j,  9.45625070e-33+0.j,  2.96890207e-33+0.j,
                -3.96989496e-03+0.j],
               [ 1.04549375e-22+0.j, -6.20346398e-35+0.j,  3.24933323e-19+0.j,
                -2.88410981e-33+0.j],
               [-1.83357261e-19+0.j,  2.87471359e-20+0.j,  1.77084911e-35+0.j,
                -9.48149097e-33+0.j],
               [-6.65529046e-20+0.j,  1.77637936e-19+0.j, -2.92843154e-17+0.j,
                 4.44395225e-35+0.j]]),
```

```
            array([[ 0.00000000e+00+0.j,  1.91732598e-22+0.j,  3.18745339e-03+0.j,
                     0.00000000e+00+0.j],
                   [-1.99250917e-20+0.j,  7.99076110e-33+0.j, -2.99910149e-17+0.j,
                    -3.19298092e-03+0.j],
                   [ 3.49446031e-17+0.j,  9.25454122e-17+0.j,  2.72933783e-33+0.j,
                    -1.92172772e-22+0.j],
                   [ 1.26840383e-17+0.j, -2.46848055e-17+0.j,  0.00000000e+00+0.j,
                    -1.07648747e-32+0.j]]),
            array([[ 2.00964806e-02+0.j, -1.20360611e-75+0.j,  1.91145984e-47+0.j,
                     0.00000000e+00+0.j],
                   [-4.29396365e-15+0.j,  2.00616905e-02+0.j,  0.00000000e+00+0.j,
                    -1.91563030e-47+0.j],
                   [-1.56225289e-17+0.j,  4.48495721e-63+0.j, -2.01426594e-02+0.j,
                     1.08880920e-75+0.j],
                   [ 6.42427665e-81+0.j,  2.58707001e-63+0.j,  0.00000000e+00+0.j,
                    -2.01077894e-02+0.j]]),
            array([[ 0.00000000e+00+0.j, -5.86730121e-02+0.j, -4.79549514e-21+0.j,
                     0.00000000e+00+0.j],
                   [-7.38724537e-18+0.j, -7.84223607e-31+0.j,  0.00000000e+00+0.j,
                    -4.78720075e-21+0.j],
                   [ 8.39560813e-18+0.j, -1.84033031e-17+0.j, -8.02376203e-32+0.j,
                    -5.85384991e-02+0.j],
                   [-1.74688604e-17+0.j,  1.15844754e-40+0.j,  0.00000000e+00+0.j,
                     8.64437858e-31+0.j]]),
            array([[ 4.71089270e-02+0.j, -4.68163568e-76+0.j,  3.64527878e-50+0.j,
                     0.00000000e+00+0.j],
                   [-1.01436419e-15+0.j, -4.71906210e-02+0.j,  0.00000000e+00+0.j,
                    -1.86617887e-50+0.j],
                   [ 1.04573628e-17+0.j,  4.36917936e-66+0.j,  4.70009256e-02+0.j,
                    -4.64481666e-76+0.j],
                   [-4.29970982e-80+0.j,  2.52028557e-66+0.j,  0.00000000e+00+0.j,
                    -4.70824324e-02+0.j]]),
            array([[ 0.00000000e+00+0.j, -3.34378582e-18+0.j, -6.75725151e-02+0.j,
                     0.00000000e+00+0.j],
                   [-8.35071473e-22+0.j, -8.36929429e-34+0.j,  0.00000000e+00+0.j,
                    -6.74555369e-02+0.j],
                   [-3.60237924e-18+0.j, -8.45061378e-18+0.j, -3.07014480e-34+0.j,
                     3.36252135e-18+0.j],
                   [-6.35147907e-18+0.j,  3.73548404e-18+0.j,  0.00000000e+00+0.j,
                     1.14812503e-33+0.j]]),
            array([[ 9.98687129e-01+0.j,  4.62407006e-77+0.j, -3.85501467e-49+0.j,
                     0.00000000e+00+0.j],
                   [ 2.21307400e-16+0.j,  9.96958252e-01+0.j,  0.00000000e+00+0.j,
                     3.85499935e-49+0.j],
                   [ 0.00000000e+00+0.j, -9.02549263e-65+0.j,  9.96397551e-01+0.j,
                     4.00081196e-80+0.j],
                   [ 1.89840017e-81+0.j, -5.20619936e-65+0.j,  0.00000000e+00+0.j,
                     9.94672637e-01+0.j]])]}]],
    'probabilities': [1.0],
    'gate_qubits': [[0, 1]]},
   {'type': 'qerror',
    'operations': ['cx'],
    'instructions': [[{'name': 'kraus',
       'qubits': [0, 1],
       'params': [array([[ 0.00000000e+00+0.j,  1.01714713e-30+0.j,  1.40116225e-
03+0.j,
                     0.00000000e+00+0.j],
                   [ 3.89791127e-20+0.j,  1.56428512e-33+0.j,  4.40151196e-16+0.j,
                    -1.40498701e-03+0.j],
                   [ 2.73453517e-17+0.j, -2.46513350e-16+0.j, -1.18192275e-33+0.j,
                    -1.03408189e-30+0.j],
                   [-2.04448318e-17+0.j,  9.56754820e-17+0.j,  0.00000000e+00+0.j,
                    -3.87554334e-34+0.j]]),
            array([[ 1.12458215e-03+0.j, -2.16687641e-22+0.j,  0.00000000e+00+0.j,
```

```
                0.00000000e+00+0.j],
               [-3.10379812e-16+0.j, -1.12765194e-03+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 2.50306300e-19+0.j,  0.00000000e+00+0.j, -1.12690033e-03+0.j,
                2.17147166e-22+0.j],
               [-1.98253853e-36+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                1.12997644e-03+0.j]]),
        array([[ 0.00000000e+00+0.j, -3.13117873e-34+0.j,  4.77719001e-32+0.j,
                4.71247600e-03+0.j],
               [-1.17945438e-23+0.j, -5.03554801e-37+0.j, -2.45632071e-19+0.j,
               -4.78916766e-32+0.j],
               [-8.27432764e-21+0.j,  7.48699896e-20+0.j,  1.18655339e-36+0.j,
                3.15288645e-34+0.j],
               [ 6.18632516e-21+0.j, -6.64170507e-20+0.j,  8.46997079e-18+0.j,
               -6.81226060e-37+0.j]]),
        array([[ 0.00000000e+00+0.j,  3.78226463e-03+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-2.94869120e-17+0.j, -1.79463292e-25+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 3.98268976e-17+0.j,  1.96172759e-17+0.j, -7.26282852e-22+0.j,
               -3.79006127e-03+0.j],
               [ 2.06267980e-16+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                7.29148208e-22+0.j]]),
        array([[-2.18848146e-02-0.j, -4.19746512e-21+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-5.68741617e-15+0.j,  2.19445538e-02+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 7.29155919e-18+0.j,  0.00000000e+00+0.j, -2.18397948e-02+0.j,
                4.20835488e-21+0.j],
               [ 2.07576244e-35+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                2.18994111e-02+0.j]]),
        array([[ 0.00000000e+00+0.j,  2.46116218e-32+0.j,  6.39246881e-02+0.j,
                0.00000000e+00+0.j],
               [ 2.75225623e-21+0.j, -3.48040877e-35+0.j, -1.00231149e-17+0.j,
                6.37506673e-02+0.j],
               [ 1.93081384e-18+0.j,  1.99645075e-18+0.j, -1.81707660e-34+0.j,
               -2.44110419e-32+0.j],
               [-1.44357825e-18+0.j, -9.17783854e-18+0.j,  0.00000000e+00+0.j,
                2.17600491e-34+0.j]]),
        array([[-5.13063806e-02-0.j, -9.51510480e-24+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-7.80115058e-16+0.j, -5.11667104e-02+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  5.14121419e-02+0.j,
                4.87005767e-24+0.j],
               [-1.38900477e-34+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                5.12721837e-02+0.j]]),
        array([[ 0.00000000e+00+0.j, -7.36043695e-02+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 5.30853731e-17+0.j,  3.33426131e-24+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-1.89365593e-17+0.j,  0.00000000e+00+0.j, -3.75902814e-24+0.j,
               -7.34529559e-02+0.j],
               [ 1.10706880e-17+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                4.14059884e-25+0.j]]),
        array([[ 9.98442510e-01+0.j, -9.22489253e-23+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-2.21169612e-16+0.j,  9.95724472e-01+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  9.96388587e-01+0.j,
                9.22484078e-23+0.j],
               [-6.68037906e-36+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                9.93676140e-01+0.j]])]}]],
 'probabilities': [1.0],
```

```
     'gate_qubits': [[1, 0]]},
    {'type': 'qerror',
     'operations': ['cx'],
     'instructions': [[{'name': 'kraus',
        'qubits': [0, 1],
        'params': [array([[ 5.87721417e-04+0.j,  0.00000000e+00+0.j,  0.00000000e+
00+0.j,
                  0.00000000e+00+0.j],
                 [-2.81032236e-13+0.j, -5.89393439e-04+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-6.42418761e-17+0.j,  0.00000000e+00+0.j, -5.89094222e-04+0.j,
                  0.00000000e+00+0.j],
                 [-3.20228089e-17+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  5.90770150e-04+0.j]]),
         array([[ 0.00000000e+00+0.j,  1.29057558e-30+0.j,  1.52507080e-03+0.j,
                  0.00000000e+00+0.j],
                 [ 7.89814859e-23+0.j,  1.49801342e-34+0.j, -4.68311953e-16+0.j,
                 -1.52940951e-03+0.j],
                 [ 6.82884522e-19+0.j,  4.27654023e-17+0.j, -2.75377387e-35+0.j,
                 -1.28990614e-30+0.j],
                 [-2.06543576e-18+0.j,  3.01241145e-16+0.j,  0.00000000e+00+0.j,
                 -1.22677030e-34+0.j]]),
         array([[ 0.00000000e+00+0.j,  1.97679561e-03+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [ 1.47847720e-19+0.j, -4.15500458e-32+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-7.25920182e-17+0.j, -1.13108591e-16+0.j,  1.09170787e-31+0.j,
                 -1.98141303e-03+0.j],
                 [ 1.12731951e-16+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -6.78189899e-32+0.j]]),
         array([[ 0.00000000e+00+0.j, -6.79835527e-32+0.j,  5.14823643e-32+0.j,
                 -5.12956168e-03+0.j],
                 [-3.82664079e-24+0.j, -5.20021658e-36+0.j,  3.39332508e-19+0.j,
                 -5.15618427e-32+0.j],
                 [-3.30856496e-20+0.j, -2.26388595e-18+0.j,  8.32154270e-38+0.j,
                  6.83423619e-32+0.j],
                 [ 1.00070043e-19+0.j,  3.32247816e-19+0.j, -3.18012345e-17+0.j,
                  5.13468058e-36+0.j]]),
         array([[-2.23597491e-02-0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [ 8.12956982e-15+0.j,  2.24233609e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [ 1.24153479e-18+0.j,  0.00000000e+00+0.j, -2.23076427e-02+0.j,
                  0.00000000e+00+0.j],
                 [-4.96613917e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  2.23711062e-02+0.j]]),
         array([[ 2.62691458e-02+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-2.69996645e-15+0.j,  2.61946241e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [ 0.00000000e+00+0.j,  0.00000000e+00+0.j, -2.63305055e-02+0.j,
                  0.00000000e+00+0.j],
                 [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -2.62558098e-02+0.j]]),
         array([[ 0.00000000e+00+0.j,  2.29688982e-31+0.j,  6.81654710e-02+0.j,
                  0.00000000e+00+0.j],
                 [ 2.49330083e-22+0.j,  5.62323069e-34+0.j,  1.06874391e-17+0.j,
                  6.79720956e-02+0.j],
                 [ 2.15574133e-18+0.j,  1.66404877e-19+0.j, -5.04089396e-34+0.j,
                 -2.30051806e-31+0.j],
                 [-6.52020233e-18+0.j,  1.54424549e-18+0.j,  0.00000000e+00+0.j,
                 -5.82439131e-35+0.j]]),
         array([[ 0.00000000e+00+0.j, -7.52068116e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
```

```
                [-1.34161444e-17+0.j,  3.56201591e-30+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 3.87689629e-18+0.j,  0.00000000e+00+0.j, -1.05162237e-29+0.j,
                 -7.50315522e-02+0.j],
                [-7.36777735e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  6.97564550e-30+0.j]]),
        array([[-9.99404637e-01-0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-4.42679627e-16+0.j, -9.96569473e-01+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 0.00000000e+00+0.j,  0.00000000e+00+0.j, -9.97075659e-01+0.j,
                  0.00000000e+00+0.j],
                [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -9.94247102e-01+0.j]])]}]],
    'probabilities': [1.0],
    'gate_qubits': [[1, 2]]},
   {'type': 'qerror',
    'operations': ['cx'],
    'instructions': [[{'name': 'kraus',
       'qubits': [0, 1],
       'params': [array([[ 9.49507494e-04+0.j,  0.00000000e+00+0.j,  0.00000000e+
00+0.j,
                  0.00000000e+00+0.j],
                [-1.45336561e-13+0.j, -9.54648413e-04+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 1.10135546e-17+0.j,  0.00000000e+00+0.j, -9.53048683e-04+0.j,
                  0.00000000e+00+0.j],
                [ 2.15604893e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  9.58208775e-04+0.j]]),
        array([[ 0.00000000e+00+0.j, -1.75142045e-30+0.j, -2.64470712e-03+0.j,
                  0.00000000e+00+0.j],
                [-2.57514827e-21+0.j, -5.56461779e-33+0.j, -9.40265552e-16+0.j,
                  2.65902636e-03+0.j],
                [ 1.30092439e-18+0.j,  2.84063941e-17+0.j,  5.35157015e-33+0.j,
                  1.77966399e-30+0.j],
                [-1.80720097e-16+0.j, -7.99176205e-16+0.j,  0.00000000e+00+0.j,
                  2.04200005e-34+0.j]]),
        array([[ 0.00000000e+00+0.j, -3.19851115e-03+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-2.76587931e-19+0.j,  3.18677818e-30+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 1.60900254e-16+0.j,  1.06613918e-15+0.j, -3.23077712e-30+0.j,
                  3.21044000e-03+0.j],
                [ 2.31421771e-16+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  4.95288659e-32+0.j]]),
        array([[ 0.00000000e+00+0.j, -2.31633468e-34+0.j,  1.59116751e-31+0.j,
                  8.90896096e-03+0.j],
                [-3.29443011e-25+0.j, -1.80670555e-36+0.j, -6.21360271e-19+0.j,
                 -1.60261706e-31+0.j],
                [ 1.66429427e-22+0.j,  3.52098231e-21+0.j,  1.76813988e-36+0.j,
                  2.35518613e-34+0.j],
                [-2.31198233e-20+0.j, -2.70386349e-19+0.j,  3.45396902e-17+0.j,
                  3.56865146e-38+0.j]]),
        array([[-3.07226330e-02-0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 1.88100139e-15+0.j,  3.08889745e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 0.00000000e+00+0.j,  0.00000000e+00+0.j, -3.06084786e-02+0.j,
                  0.00000000e+00+0.j],
                [ 6.82759124e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  3.07742021e-02+0.j]]),
        array([[ 3.08764525e-02+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-1.46595920e-15+0.j,  3.07101784e-02+0.j,  0.00000000e+00+0.j,
```

```
                    0.00000000e+00+0.j],
                  [ 0.00000000e+00+0.j,  0.00000000e+00+0.j, -3.09916063e-02+0.j,
                    0.00000000e+00+0.j],
                  [ 6.85027662e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                   -3.08247121e-02+0.j]]),
          array([[ 0.00000000e+00+0.j,  5.02917085e-30+0.j, -8.60016107e-02+0.j,
                    0.00000000e+00+0.j],
                  [ 7.53811788e-23+0.j, -2.95978043e-34+0.j,  2.69334784e-17+0.j,
                   -8.55384797e-02+0.j],
                  [-3.80813857e-20+0.j,  2.91487529e-17+0.j,  3.06373199e-34+0.j,
                   -5.04623859e-30+0.j],
                  [ 5.29013965e-18+0.j,  2.25067050e-17+0.j,  0.00000000e+00+0.j,
                   -1.09341140e-35+0.j]]),
          array([[ 0.00000000e+00+0.j, -1.03492268e-01+0.j,  0.00000000e+00+0.j,
                    0.00000000e+00+0.j],
                  [ 1.14935831e-18+0.j, -1.25814195e-29+0.j,  0.00000000e+00+0.j,
                    0.00000000e+00+0.j],
                  [-6.30318789e-17+0.j, -3.24381657e-17+0.j,  1.28059182e-29+0.j,
                   -1.03107728e-01+0.j],
                  [-9.87646078e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                   -2.46796587e-31+0.j]]),
          array([[-9.99050481e-01-0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                    0.00000000e+00+0.j],
                  [-2.20826635e-16+0.j, -9.93670451e-01+0.j,  0.00000000e+00+0.j,
                    0.00000000e+00+0.j],
                  [ 0.00000000e+00+0.j,  0.00000000e+00+0.j, -9.95338365e-01+0.j,
                    0.00000000e+00+0.j],
                  [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                   -9.89978326e-01+0.j]])]}]],
    'probabilities': [1.0],
    'gate_qubits': [[1, 3]]},
   {'type': 'qerror',
    'operations': ['cx'],
    'instructions': [[{'name': 'kraus',
       'qubits': [0, 1],
       'params': [array([[ 4.99109977e-04+0.j,  0.00000000e+00+0.j,  0.00000000e+
00+0.j,
                    0.00000000e+00+0.j],
                  [-6.51971705e-17+0.j, -5.00098779e-04+0.j,  0.00000000e+00+0.j,
                    0.00000000e+00+0.j],
                  [ 0.00000000e+00+0.j,  0.00000000e+00+0.j, -5.00314323e-04+0.j,
                    0.00000000e+00+0.j],
                  [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                    5.01305511e-04+0.j]]),
          array([[ 0.00000000e+00+0.j,  1.29483571e-03+0.j,  0.00000000e+00+0.j,
                    0.00000000e+00+0.j],
                  [-7.79416682e-19+0.j,  1.04212059e-31+0.j,  0.00000000e+00+0.j,
                    0.00000000e+00+0.j],
                  [ 1.57625731e-16+0.j, -5.48074686e-16+0.j, -7.33010690e-33+0.j,
                   -1.29796014e-03+0.j],
                  [ 1.00869183e-15+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                   -9.69710147e-32+0.j]]),
          array([[ 0.00000000e+00+0.j, -2.47584965e-30+0.j,  1.67832407e-03+0.j,
                    0.00000000e+00+0.j],
                  [ 4.29425302e-21+0.j,  6.24463130e-33+0.j, -9.57497991e-17+0.j,
                   -1.68164904e-03+0.j],
                  [ 9.40706664e-18+0.j,  8.12911800e-17+0.j, -3.35253721e-33+0.j,
                    2.46338299e-30+0.j],
                  [ 4.90086330e-17+0.j,  1.38393810e-16+0.j,  0.00000000e+00+0.j,
                   -2.90294110e-33+0.j]]),
          array([[ 0.00000000e+00+0.j,  1.36719006e-32+0.j, -5.73173301e-32+0.j,
                    4.35405830e-03+0.j],
                  [-2.43041374e-23+0.j,  1.31176367e-35+0.j, -6.03891804e-19+0.j,
                    5.74658921e-32+0.j],
```

```
                [-5.32410733e-20+0.j, -4.60366595e-19+0.j, -1.58049137e-35+0.j,
                 -1.36390178e-32+0.j],
                [-2.77373630e-19+0.j,  1.31431648e-18+0.j, -1.58425598e-17+0.j,
                  2.69548390e-36+0.j]]),
        array([[ 2.06070380e-02+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 9.65679203e-16+0.j,  2.05662934e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-1.02975270e-17+0.j,  0.00000000e+00+0.j, -2.06567625e-02+0.j,
                  0.00000000e+00+0.j],
                [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -2.06159196e-02+0.j]]),
        array([[-2.42081192e-02-0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-2.01530149e-15+0.j,  2.42560786e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-5.37413732e-18+0.j,  0.00000000e+00+0.j, -2.41498459e-02+0.j,
                  0.00000000e+00+0.j],
                [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  2.41976899e-02+0.j]]),
        array([[ 0.00000000e+00+0.j,  6.28028665e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-1.83664399e-17+0.j,  4.27307776e-31+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-1.80641197e-17+0.j,  9.84876971e-18+0.j,  7.66931575e-31+0.j,
                  6.26516889e-02+0.j],
                [ 2.54708987e-17+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -1.19524848e-30+0.j]]),
        array([[ 0.00000000e+00+0.j, -3.59285217e-30+0.j,  6.92939218e-02+0.j,
                  0.00000000e+00+0.j],
                [-1.23929156e-22+0.j, -1.46723564e-34+0.j,  1.08690359e-17+0.j,
                  6.91569129e-02+0.j],
                [-2.71481402e-19+0.j, -1.96980549e-17+0.j,  8.61252921e-35+0.j,
                  3.60137005e-30+0.j],
                [-1.41435507e-18+0.j, -6.18187025e-18+0.j,  0.00000000e+00+0.j,
                  6.08402550e-35+0.j]]),
        array([[-0.99949441-0.j,  0.        +0.j,  0.        +0.j,
                  0.        +0.j],
                [ 0.        +0.j, -0.99751819+0.j,  0.        +0.j,
                  0.        +0.j],
                [ 0.        +0.j,  0.        +0.j, -0.99708845+0.j,
                  0.        +0.j],
                [ 0.        +0.j,  0.        +0.j,  0.        +0.j,
                 -0.99511699+0.j]])]}]],
    'probabilities': [1.0],
    'gate_qubits': [[2, 1]]},
   {'type': 'qerror',
    'operations': ['cx'],
    'instructions': [[{'name': 'kraus',
       'qubits': [0, 1],
       'params': [array([[ 1.02443802e-03+0.j,  0.00000000e+00+0.j,  0.00000000e+
00+0.j,
                  0.00000000e+00+0.j],
                [ 1.64413852e-13+0.j, -1.02856475e-03+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [-6.40071723e-18+0.j,  0.00000000e+00+0.j, -1.03042939e-03+0.j,
                  0.00000000e+00+0.j],
                [ 2.45920414e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  1.03458026e-03+0.j]]),
        array([[ 0.00000000e+00+0.j, -2.85395260e-03+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 6.01819611e-19+0.j, -7.49777658e-30+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 1.74292876e-16+0.j,  3.34360438e-16+0.j,  7.34283624e-30+0.j,
```

```
                2.87064379e-03+0.j],
               [ 1.55267685e-16+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                1.69053545e-31+0.j]]),
        array([[ 0.00000000e+00+0.j, -3.22652431e-30+0.j,  3.45180847e-03+0.j,
                0.00000000e+00+0.j],
               [-5.54660370e-24+0.j,  3.33654935e-35+0.j,  3.09543367e-17+0.j,
                -3.46571334e-03+0.j],
               [-5.46389437e-20+0.j, -5.55191569e-17+0.j, -3.29853446e-35+0.j,
                3.24975793e-30+0.j],
               [ 2.29951217e-19+0.j,  4.45365059e-19+0.j,  0.00000000e+00+0.j,
                -4.40093190e-37+0.j]]),
        array([[ 0.00000000e+00+0.j, -4.98717211e-33+0.j, -8.16375246e-33+0.j,
                9.61629452e-03+0.j],
               [ 3.63657621e-26+0.j, -1.75986163e-37+0.j,  1.56456749e-18+0.j,
                8.36713864e-33+0.j],
               [ 3.58234870e-22+0.j, -8.51707558e-20+0.j,  1.96291624e-37+0.j,
                5.03485215e-33+0.j],
               [-1.50765254e-21+0.j, -8.77627044e-19+0.j, -4.50214669e-17+0.j,
                -2.01327566e-38+0.j]]),
        array([[-3.19085008e-02-0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-2.60262404e-15+0.j, -3.17804799e-02+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-3.54580932e-18+0.j,  0.00000000e+00+0.j,  3.20951160e-02+0.j,
                0.00000000e+00+0.j],
               [ 1.41832373e-17+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                3.19663464e-02+0.j]]),
        array([[ 3.20725985e-02+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 4.90592370e-15+0.j, -3.22017961e-02+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  3.18861143e-02+0.j,
                0.00000000e+00+0.j],
               [ 1.42303806e-17+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                -3.20145606e-02+0.j]]),
        array([[ 0.00000000e+00+0.j,  8.93501354e-02+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-1.98376040e-17+0.j,  2.49289837e-28+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [-7.05949763e-18+0.j,  1.39880720e-17+0.j, -2.43691402e-28+0.j,
                8.88306142e-02+0.j],
               [-7.38237567e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                -6.07499515e-30+0.j]]),
        array([[ 0.00000000e+00+0.j, -1.32704007e-30+0.j, -1.07514589e-01+0.j,
                0.00000000e+00+0.j],
               [ 9.97840233e-22+0.j, -3.09250617e-33+0.j, -6.73878188e-17+0.j,
                -1.07083227e-01+0.j],
               [ 9.82960743e-18+0.j, -8.72718838e-18+0.j,  4.32744886e-33+0.j,
                1.33742563e-30+0.j],
               [-4.13684818e-17+0.j, -1.42969511e-19+0.j,  0.00000000e+00+0.j,
                -1.23752942e-33+0.j]]),
        array([[-9.98975549e-01-0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 4.41459917e-16+0.j, -9.94967535e-01+0.j,  0.00000000e+00+0.j,
                0.00000000e+00+0.j],
               [ 0.00000000e+00+0.j,  0.00000000e+00+0.j, -9.93167063e-01+0.j,
                0.00000000e+00+0.j],
               [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                -9.89182354e-01+0.j]])]}]],
 'probabilities': [1.0],
 'gate_qubits': [[3, 1]]},
{'type': 'qerror',
 'operations': ['cx'],
 'instructions': [[{'name': 'kraus',
```

```
        'qubits': [0, 1],
        'params': [array([[ 0.00000000e+00+0.j, -3.54865405e-32+0.j, -2.13565202e-
03+0.j,
                  0.00000000e+00+0.j],
                 [-4.49867245e-21+0.j,  1.12144922e-34+0.j,  0.00000000e+00+0.j,
                  2.14136540e-03+0.j],
                 [-3.36803859e-19+0.j, -1.73066709e-16+0.j, -3.48410359e-34+0.j,
                  3.79825557e-32+0.j],
                 [-5.75049709e-18+0.j, -4.10578541e-16+0.j,  0.00000000e+00+0.j,
                  2.36020835e-34+0.j]]),
         array([[ 1.72261932e-03+0.j, -7.04335751e-22+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-7.36618507e-17+0.j, -1.72722773e-03+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-1.91827736e-18+0.j,  0.00000000e+00+0.j, -1.72841784e-03+0.j,
                  7.05783811e-22+0.j],
                 [ 8.03444457e-39+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  1.73304177e-03+0.j]]),
         array([[ 0.00000000e+00+0.j,  1.22497168e-34+0.j, -1.57242906e-31+0.j,
                  5.94457602e-03+0.j],
                 [ 1.47186506e-23+0.j, -2.46115441e-37+0.j,  0.00000000e+00+0.j,
                  1.57825391e-31+0.j],
                 [ 1.10194687e-21+0.j,  5.90751721e-19+0.j,  5.91700941e-37+0.j,
                 -1.31378818e-34+0.j],
                 [ 1.88143407e-20+0.j,  1.39204663e-18+0.j,  6.29561724e-19+0.j,
                 -3.46344575e-37+0.j]]),
         array([[ 0.00000000e+00+0.j, -4.79490169e-03+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-1.10167263e-16+0.j, -5.15485227e-27+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-6.80295900e-17+0.j, -1.50822408e-18+0.j,  1.95266205e-21+0.j,
                  4.81104184e-03+0.j],
                 [ 3.38595116e-17+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -1.96058121e-21+0.j]]),
         array([[ 2.61514909e-02+0.j,  1.06190368e-20+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [ 1.58471715e-15+0.j, -2.62214524e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [ 3.19265360e-17+0.j,  0.00000000e+00+0.j,  2.60637575e-02+0.j,
                 -1.06449762e-20+0.j],
                 [ 7.19599120e-38+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -2.61334843e-02+0.j]]),
         array([[ 0.00000000e+00+0.j, -7.27925356e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-2.05329879e-17+0.j,  5.70506888e-24+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [ 1.29749256e-17+0.j,  0.00000000e+00+0.j,  1.35756848e-23+0.j,
                 -7.25483302e-02+0.j],
                 [-1.97862334e-18+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -1.94051120e-23+0.j]]),
         array([[ 0.00000000e+00+0.j, -2.83590393e-32+0.j, -8.14600515e-02+0.j,
                  0.00000000e+00+0.j],
                 [-6.99321440e-21+0.j,  4.00004120e-35+0.j,  0.00000000e+00+0.j,
                 -8.12427078e-02+0.j],
                 [-5.23564371e-19+0.j,  6.71617652e-18+0.j,  4.71516902e-35+0.j,
                  2.80505487e-32+0.j],
                 [-8.93918601e-18+0.j,  1.40144607e-17+0.j,  0.00000000e+00+0.j,
                 -8.86181860e-35+0.j]]),
         array([[ 6.57057689e-02+0.j,  2.35846332e-23+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-1.21136045e-15+0.j,  6.55304592e-02+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                 [-1.45947042e-17+0.j,  0.00000000e+00+0.j, -6.59269415e-02+0.j,
                 -1.32604912e-23+0.j],
```

```
                [-4.87878271e-37+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                 -6.57510418e-02+0.j]]),
        array([[ 9.97494804e-01+0.j, -2.78738281e-22+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 0.00000000e+00+0.j,  9.94833386e-01+0.j,  0.00000000e+00+0.j,
                  0.00000000e+00+0.j],
                [ 0.00000000e+00+0.j,  0.00000000e+00+0.j,  9.94148395e-01+0.j,
                  2.78735780e-22+0.j],
                [ 3.02364659e-38+0.j,  0.00000000e+00+0.j,  0.00000000e+00+0.j,
                  9.91495906e-01+0.j]])]}]],
    'probabilities': [1.0],
    'gate_qubits': [[3, 4]]},
   {'type': 'qerror',
    'operations': ['cx'],
    'instructions': [[{'name': 'kraus',
       'qubits': [0, 1],
       'params': [array([[ 0.00000000e+00+0.j, -1.89934634e-03+0.j, -1.36572304e-
18+0.j,
                  0.00000000e+00+0.j],
                [ 2.97721910e-19+0.j, -4.62315938e-32+0.j,  0.00000000e+00+0.j,
                 -1.36133119e-18+0.j],
                [ 2.95844722e-16+0.j,  5.34290327e-16+0.j,  5.77211852e-33+0.j,
                  1.90386160e-03+0.j],
                [-7.20527097e-17+0.j,  6.90676297e-42+0.j,  0.00000000e+00+0.j,
                  4.06120457e-32+0.j]]),
        array([[ 1.53267666e-03+0.j, -6.08952429e-77+0.j,  8.99373170e-49+0.j,
                  0.00000000e+00+0.j],
                [ 1.70277202e-16+0.j, -1.53725726e-03+0.j,  0.00000000e+00+0.j,
                 -9.01014853e-49+0.j],
                [-6.82473755e-19+0.j,  3.06919622e-64+0.j, -1.53632025e-03+0.j,
                  3.79938817e-77+0.j],
                [-1.49266286e-80+0.j,  2.75639693e-66+0.j,  0.00000000e+00+0.j,
                  1.54091174e-03+0.j]]),
        array([[ 0.00000000e+00+0.j, -1.54738023e-34+0.j, -3.68444890e-34+0.j,
                  5.28582731e-03+0.j],
                [-1.09961177e-22+0.j,  5.47580495e-36+0.j, -1.44997927e-18+0.j,
                  3.99693284e-34+0.j],
                [-3.65838510e-20+0.j,  1.84507934e-20+0.j, -3.06519451e-36+0.j,
                  1.57266954e-34+0.j],
                [-1.46849152e-20+0.j,  1.54021573e-18+0.j, -2.07008745e-20+0.j,
                 -2.39890253e-36+0.j]]),
        array([[ 0.00000000e+00+0.j, -3.62206944e-22+0.j,  4.26539594e-03+0.j,
                  0.00000000e+00+0.j],
                [ 2.00257381e-20+0.j, -6.39297322e-34+0.j, -4.19192562e-19+0.j,
                 -4.27814363e-03+0.j],
                [ 6.66240979e-18+0.j,  1.76158882e-17+0.j, -2.05702247e-34+0.j,
                  3.63066899e-22+0.j],
                [ 2.67454095e-18+0.j,  1.85824454e-18+0.j,  0.00000000e+00+0.j,
                  8.55881506e-34+0.j]]),
        array([[-2.46700328e-02-0.j, -8.59965757e-76+0.j,  1.43877762e-47+0.j,
                  0.00000000e+00+0.j],
                [ 2.66964646e-15+0.j, -2.45965229e-02+0.j,  0.00000000e+00+0.j,
                 -1.44189797e-47+0.j],
                [-1.09523957e-17+0.j,  4.91164799e-63+0.j,  2.47286802e-02+0.j,
                  7.22149564e-76+0.j],
                [ 1.41752857e-79+0.j,  4.41107394e-65+0.j,  0.00000000e+00+0.j,
                  2.46549955e-02+0.j]]),
        array([[ 0.00000000e+00+0.j, -4.92167474e-17+0.j,  6.86560057e-02+0.j,
                  0.00000000e+00+0.j],
                [ 1.04676768e-20+0.j,  1.77375239e-34+0.j,  0.00000000e+00+0.j,
                  6.84514298e-02+0.j],
                [-1.52474452e-17+0.j, -3.15138376e-18+0.j,  1.46157363e-33+0.j,
                  4.90952211e-17+0.j],
                [ 3.12108601e-17+0.j, -1.45965604e-18+0.j,  0.00000000e+00+0.j,
```

```
                -1.65558265e-33+0.j]]),
        array([[ 0.00000000e+00+0.j, -7.68183903e-02+0.j, -1.33523797e-19+0.j,
                 0.00000000e+00+0.j],
               [-1.82312692e-17+0.j, -9.83102877e-31+0.j,  0.00000000e+00+0.j,
                -1.33125913e-19+0.j],
               [-1.27570495e-17+0.j, -1.20469037e-17+0.j, -2.07421821e-32+0.j,
                -7.66362052e-02+0.j],
               [ 1.53002782e-17+0.j, -9.56210631e-42+0.j,  0.00000000e+00+0.j,
                 1.00781125e-30+0.j]]),
        array([[-6.19885651e-02-0.j, -2.90189448e-76+0.j,  2.83766562e-50+0.j,
                 0.00000000e+00+0.j],
               [ 1.37684733e-16+0.j,  6.21738259e-02+0.j,  0.00000000e+00+0.j,
                -1.59583907e-50+0.j],
               [ 0.00000000e+00+0.j,  5.43602940e-66+0.j, -6.18415509e-02+0.j,
                -2.87399029e-76+0.j],
               [-9.60970946e-79+0.j,  4.88201265e-68+0.j,  0.00000000e+00+0.j,
                 6.20263724e-02+0.j]]),
        array([[ 9.97770744e-01+0.j, -3.91978602e-77+0.j,  3.56121377e-49+0.j,
                 0.00000000e+00+0.j],
               [ 0.00000000e+00+0.j,  9.94797663e-01+0.j,  0.00000000e+00+0.j,
                -3.56118855e-49+0.j],
               [ 0.00000000e+00+0.j,  1.21307505e-64+0.j,  9.95404397e-01+0.j,
                -5.83624406e-80+0.j],
               [-5.61745121e-80+0.j,  1.08944366e-66+0.j,  0.00000000e+00+0.j,
                 9.92438367e-01+0.j]])]}]],
    'probabilities': [1.0],
    'gate_qubits': [[4, 3]]},
   {'type': 'roerror',
    'operations': ['measure'],
    'probabilities': [[0.984, 0.016000000000000014], [0.042, 0.958]],
    'gate_qubits': [[0]]},
   {'type': 'roerror',
    'operations': ['measure'],
    'probabilities': [[0.979, 0.021], [0.07899999999999996, 0.921]],
    'gate_qubits': [[1]]},
   {'type': 'roerror',
    'operations': ['measure'],
    'probabilities': [[0.991, 0.009000000000000008], [0.043, 0.957]],
    'gate_qubits': [[2]]},
   {'type': 'roerror',
    'operations': ['measure'],
    'probabilities': [[0.968, 0.03200000000000003], [0.05, 0.95]],
    'gate_qubits': [[3]]},
   {'type': 'roerror',
    'operations': ['measure'],
    'probabilities': [[0.985, 0.015], [0.039000000000000035, 0.961]],
    'gate_qubits': [[4]]}],
  'x90_gates': []},
 'coupling_map': [[0, 1],
  [1, 0],
  [1, 2],
  [1, 3],
  [2, 1],
  [3, 1],
  [3, 4],
  [4, 3]],
 'tot_q_device': 5,
 'basis_gates': ['u1', 'u2', 'u3', 'cx', 'id']}
```

In [284]:
```
print(dir(cirq.google.Sycamore))
cirq.google.Sycamore23.gate_definitions
```

```
['__abstractmethods__', '__class__', '__delattr__', '__dict__', '__dir__', '__doc
__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__',
'__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__n
ew__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '_
_str__', '__subclasshook__', '__weakref__', '_abc_impl', '_create_target_set', '_
find_operation_type', '_qid_from_str', 'can_add_operation_into_moment', 'decompos
e_operation', 'duration_of', 'from_proto', 'gate_definitions', 'qubit_set', 'qubi
ts', 'validate_circuit', 'validate_moment', 'validate_operation']
```

Out[284]:
```
{cirq.ops.fsim_gate.FSimGate: [<cirq.google.devices.serializable_device._GateDefi
nition at 0x7f879e377450>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e3777d0>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e377cd0>],
 cirq.ops.swap_gates.ISwapPowGate: [<cirq.google.devices.serializable_device._Gat
eDefinition at 0x7f879e377850>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e3778d0>],
 cirq.ops.phased_x_gate.PhasedXPowGate: [<cirq.google.devices.serializable_devic
e._GateDefinition at 0x7f879e377950>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e377d50>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e377dd0>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e377e50>],
 cirq.ops.common_gates.XPowGate: [<cirq.google.devices.serializable_device._GateD
efinition at 0x7f879e3779d0>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e377ed0>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e377f50>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e377fd0>],
 cirq.ops.common_gates.YPowGate: [<cirq.google.devices.serializable_device._GateD
efinition at 0x7f879e377a50>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e374090>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e374110>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e374190>],
 cirq.ops.common_gates.ZPowGate: [<cirq.google.devices.serializable_device._GateD
efinition at 0x7f879e377ad0>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e374210>],
 cirq.ops.phased_x_z_gate.PhasedXZGate: [<cirq.google.devices.serializable_devic
e._GateDefinition at 0x7f879e377b50>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e374290>],
 cirq.ops.measurement_gate.MeasurementGate: [<cirq.google.devices.serializable_de
vice._GateDefinition at 0x7f879e377bd0>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e374310>],
 cirq.ops.wait_gate.WaitGate: [<cirq.google.devices.serializable_device._GateDefi
nition at 0x7f879e377c50>,
  <cirq.google.devices.serializable_device._GateDefinition at 0x7f879e374390>]}
```

Comment

Currently we still need to see how to find a noise backend configuration in `Cirq`. In `Qiskit` the noise model is stored as a dictionary containing information about different types of errors, the error operator applied on each register, the gate replacements, and a discrete probability distribution of each operator being applied (as shown in the above `Qiskit` code). The `Cirq` API doesn't seem to allow for access to the probability distribution and redefining it, though we may be able to gain more insight by looking into the calibration metrics for quantum engines on Google Cloud (https://cirq.readthedocs.io/en/stable/google/calibration.html). The `cirq` devices also don't seem to have an attribute defining the noise model. We should be able to create a custom noise model via implementing `cirq.NoiseModel` (https://cirq.readthedocs.io/en/stable/_modules/cirq/devices/noise_model.html#NoiseModel) by specifying noisy moments and operators. The main difficulty seems to be gaining access to the probability distributions and tuning them.

In [ ]:

```

```